

CITYBITE BANGKOK

DIGITAL ORDERING SYSTEM



PRESENTED BY

TEAM 03

THE PROBLEM

Orders arrive by walk-in, phone, and chat — the same order lands in three places, or nowhere.

STAFF

Staff reconcile channels by hand during rush → missed / wrong items

CUSTOMER

Customers can't tell if an order was received → more phone calls

OUR USERS

Customer - Staff - Owner

WHAT WE DELIVERED

CUSTOMER

Browse menu by category
→ Customize with options →
Cart → submit order → Get
a reference number

STAFF

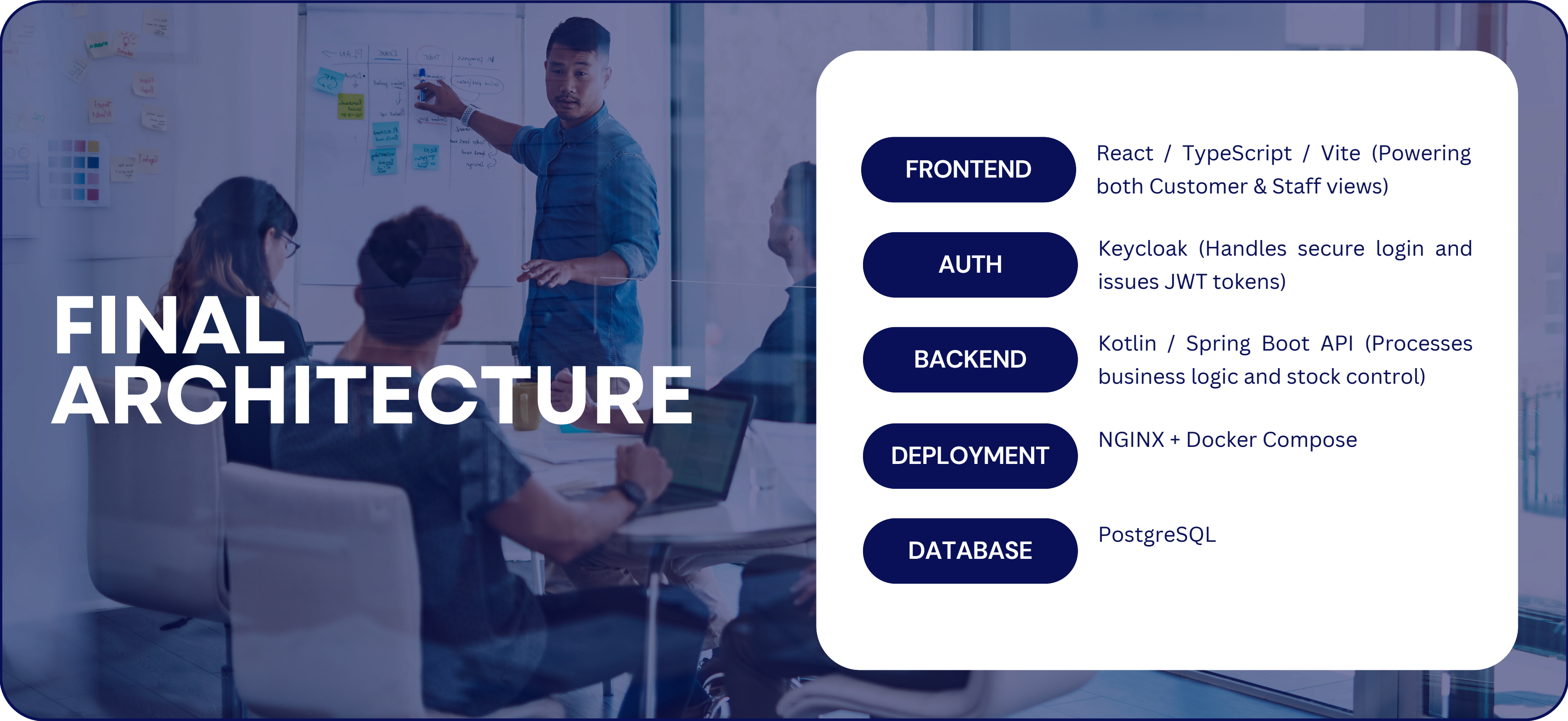
One order inbox → Full
order detail → Claim /
Complete / Cancel →
Toggle menu availability →
Manage stock

OWNER

- Some staff actions
- Manage staff roster
- Store open/closed

Smart core: Accepting an order auto-deducts ingredients from stock;
canceling restores them and credits the customer

**LIVE
DEMO**

A man in a blue shirt is standing and pointing at a whiteboard in a meeting room. Several people are seated around a table, looking at the whiteboard. The whiteboard has various diagrams and notes. The background is a blurred office setting.

FINAL ARCHITECTURE

FRONTEND

React / TypeScript / Vite (Powering both Customer & Staff views)

AUTH

Keycloak (Handles secure login and issues JWT tokens)

BACKEND

Kotlin / Spring Boot API (Processes business logic and stock control)

DEPLOYMENT

NGINX + Docker Compose

DATABASE

PostgreSQL

DATA FLOW

1.500+
Growth Business



Customer's Order Placement Journey



1.500+
Growth Business



STAFF ORDER FULFILLMENT & STOCK MANAGEMENT



1.500+
Growth Business



C4 DIAGRAM



Key Engineering Decisions

Monorepo Strategy (ADR 001)

- **The Choice:** Single Git repository for React frontends and Kotlin/Spring Boot backend.
- **Pros:** Simplified local Docker orchestration and synchronized full-stack Pull Requests.
- **Cons:** Requires a more complex CI/CD build configuration as the repository scales.

Identity via Keycloak (ADR 002-c)

- **The Choice:** Offloaded authentication to Keycloak: customer self-registration vs. admin-provisioned staff/owner access.
- **Pros:** Enhances system security posture; eliminates hand-rolling custom token validation.
- **Cons:** Introduces an external container dependency that must be hosted and managed separately.

Key Engineering Decisions

Deferred Stripe Payments (ADR 002-A)

- **The Choice:** Completely removed Stripe Thailand integration from the active MVP scope.
- **Pros:** Allows the team to focus completely on stabilizing core order-intake loops.
- **Cons:** The platform cannot process real commercial payments; transactions are entirely simulated.

Recipe-Based Stock Deduction (ADR 002-D)

- **The Choice:** Reinstated automatic ingredient deduction inside the core staff order-acceptance workflow.
- **Pros:** Programmatically eliminates the business issue of customers ordering out-of-stock menu items.
- **Cons:** Introduces a strict database dependency; items with missing recipe rows will deduct nothing.

How We Know It Works

Focuses on code quality, testing gaps, and team workflows

01

Development Pipeline

Our process maps every issue to a dedicated feature branch and standardized Pull Request.

02

Change Management

All changes are structurally tracked and reviewed by teammates through PRs.

03

Automated Testing

The backend utilizes Spring Boot context-load tests, though deeper automated business logic testing remains a known gap.



How We Know It Works

Focuses on code quality, testing gaps, and team workflows

04

End-to-End Verification

Manually test the entire order lifecycle, stock deductions, and refund credits end-to-end.

05

Stability & Delivery

These rigorous manual testing cycles are executed before every major code merge and live demonstration.





KNOWN ISSUES & LIMITATIONS



By Design | Offline Payments

Checkout features a simulated payment step; no live gateway is integrated, and no monetary transactions occur (ADR 002-A).



By Design | Unwired Analytics

Financial database schemas exist, but advanced owner analytics and revenue reporting are currently unwired (ADR 002-B).



By Design | Bookkeeping Only

The cancellation refund mechanism operates strictly as internal store credit bookkeeping rather than cash refunds (ADR 002-D).



By Design | Omitted Logistics

Rider dispatch, real-time map tracking, staff scheduling, and persistent customer order history logs are entirely out of scope.

Future roadmap

Core Experience & Testing

- Finalize the customer order-tracking view and reconcile seamless unauthenticated guest access workflows.
- Build a robust automated regression suite covering the multi-state order lifecycle and recipe stock deductions.

Core Experience & Testing

- Wire up live payment gateways (Stripe Thailand / PromptPay QR) to transition from simulated checkout to real commerce.
- Activate the deferred owner analytics engine on live production data while hardening local Docker production orchestration.

Handover Summary

Team3: City Bite



CITY
BITE

MAKE EVERY BITE
FEELING GOOD



final status



product at handover



issues, limitations



Product Status: Core MVP Delivery



The main end-to-end workflow—enabling a customer to browse the menu and submit a order, and allowing staff to receive, accept with automatic stock deduction, and progress that order through to completion or cancellation is fully functional and manually verified.

It operates inside a local multi-container Docker Compose development environment and has been tested exclusively against sample data.

Product at handover



MVP that makes the ordering process accurate, visible, and manageable for a single physical location. Core features like Stripe payment processing, delivery/rider tracking, and financial revenue dashboards were deliberately descoped during development to prioritize core workflow stability. Security is handled securely via Keycloak for role-based access control.

Issues, limitations, or unfinished parts



Testing Deficit: There is zero automated unit or integration test coverage for critical business logic like status transitions and stock deductions, meaning the system relies entirely on manual regression testing. Docker Compose service startup order is unmanaged, which can cause the backend to crash on boot if PostgreSQL isn't ready first. There is zero automated unit or integration test coverage for critical business logic like status transitions and stock deductions, meaning the system relies entirely on manual regression testing. Docker Compose service startup order is unmanaged, which can cause the backend to crash on boot if PostgreSQL isn't ready first.

Improvement next if the project continued



Build out an automated regression test suite covering the high-risk order lifecycles, stock deduction systems, and refund-credit ledgers. Integrate a live payment gateway and connect the Financial Record database tables to actual order data for real-time owner revenue reporting. Fix the database-to-menu DTO misalignments, establish proper container startup health checks (`depends_on`), and update the C4 architecture diagram. Introduce proper customer account registration, persisted order history, low-stock alerts, and prepare a secure production deployment pipeline.

